

Tachy-Shield Edge AI Board

User Guide



REVISION HISTORY

Revision	Date	Description
1.00	2025/12/11	First release.
1.01	2025/12/30	Updated sample application (ANPR, camera frames display). Updated firmware and Tachy Runtime API URLs.

TABLE OF CONTENTS

REVISION HISTORY

TABLE OF CONTENTS

1. Introduction

1.1 Supported RaspberryPi Models / OS Compatibility

2. Hardware Setup

3. Software Setup

3.1 Flashing the OS Image

3.2 Installing Required Dependencies

4. Sample Application

4.1 Object detection

4.1.1 Download Files

4.1.2 Set Up the Environment

4.1.3 Execute

4.2 ANPR (Automatic Number Plate Recognition)

4.2.1 Download Files

4.2.2 Set Up the Environment

4.2.3 Execute

4.3 Camera Display Example Guide

4.3.1 Check CSI lane configuration

4.3.2 Verify camera device

4.3.3 Display camera frames using the Python example

Appendix

Model Customization & Technical Support

Customer Support

1. Introduction

This document provides users with a step-by-step guide on how to install, configure, and operate the Tachy-Shield on a Raspberry Pi.

It explains the required hardware setup, software installation process, and sample applications to demonstrate NPU-accelerated inference on the device.

1.1 Supported RaspberryPi Models / OS Compatibility

The Tachy-Shield has been tested and verified on the following Raspberry Pi models:

- Raspberry Pi 4 Model B
- Raspberry Pi 3
- Raspberry Pi zero 2 W

All examples and procedures in this document are based on Raspberry Pi 4, which serves as the reference platform for this user guide.

The currently supported operating system is Raspberry Pi OS based on Linux kernel 6.6.x.

Kernel 6.12 and later versions are not yet supported, and compatibility updates will be released progressively as support is expanded.

2. Hardware Setup

The Tachy-Shield connects to the Raspberry Pi through the 40-pin GPIO header.

For detailed hardware specifications, pin information, and assembly instructions, please refer to the document below:

Hardware Specification & Assembly Guide:

1._Tachy-Shield_사용자_매뉴얼_수정_251020.pdf

* This document is also attached to the product page on Devicemart.

You can access it via the following link:

<https://www.devicemart.co.kr/goods/view?no=15797431>

3. Software Setup

The Tachy-Shield currently supports Raspberry Pi OS based on Linux kernel 6.6.x. At the moment, kernel 6.12 or later is not supported, and compatibility will be expanded in future releases.

For this guide, the following environment is used and verified:

- Raspberry Pi OS (Bookworm) – Kernel 6.6.x
Direct download:
[2024-10-22-raspbian-bookworm-arm64](https://www.raspberrypi.com/software/2024-10-22-raspbian-bookworm-arm64)
- Raspberry Pi 4 Model B

3.1 Flashing the OS Image

Follow the steps below to prepare your microSD card:

1. Download and install Raspberry Pi Imager (Windows / macOS / Linux).
<https://www.raspberrypi.com/software>
2. Launch the Imager and select:
CHOOSE OS → Use custom → Select the file “2024-10-22-raspbian-bookworm- arm64.img.xz”
3. Choose your microSD card.
4. (Optional) Advanced options:
Use the gear icon to pre-configure
5. Click Write to flash the image.
6. After flashing is complete, insert the microSD card into the Raspberry Pi and boot the system.

3.2 Installing Required Dependencies

After the Raspberry Pi boots for the first time, complete the initial setup including language, keyboard, display settings, and network configuration.

Once the system is ready and connected to the internet, proceed with installing the required dependencies and building the Tachy-Shield drivers.

Run the following commands:

```
sudo apt update
sudo apt install -y git
```

Clone the Tachy-Shield driver repository:

```
git clone https://github.com/Deeper-I/tachy-rpi-drivers.git
```

Build and install the host driver:

```
cd tachy-rpi-drivers/tachy_bs_host_driver
```

```
make
sudo make install
```

Build and install the dummy I2C sensor module:

```
cd ../i2c_dummy_sensor
make
sudo make install
```

Reboot the system to apply the installed kernel modules:

```
sudo reboot
```

After the system reboots, verify that the Tachy-Shield driver and the dummy I2C sensor module are successfully loaded.

1. Check the Tachy-Shield host driver

Run:

```
lsmod | grep tachy
```

Expected output example:

```
tachy_bs      16384  0
```

If the module tachy_bs appears in the list, the Tachy-Shield driver has been successfully installed and loaded.

2. Check the dummy sensor module

Run:

```
lsmod | grep dummy_sensor
```

Expected output example:

```
dummy_sensor      12288  1
v4l2_fwnode       20480  2 bcm2835_unicam,dummy_sensor
v4l2_async        20480  3 v4l2_fwnode,bcm2835_unicam,dummy_sensor
videodev         303104 10
v4l2_async,bcm2835_codec,v4l2_fwnode,videobuf2_v4l2,bcm2835_unicam,bcm2835_v4l2,rpivid
_hevc,v4l2_mem2mem,dummy_sensor,bcm2835_isp
mc               61440 10
v4l2_async,videodev,bcm2835_codec,videobuf2_v4l2,bcm2835_unicam,videobuf2_common,rpivid
_hevc,v4l2_mem2mem,dummy_sensor,bcm2835_isp
```

This indicates that the dummy sensor module is correctly loaded and registered with the V4L2 subsystem.

4. Sample Application

The following examples demonstrate how to run inference on the Tachy-Shield using the prepared software environment.

The steps below describe how to download the sample code, load the required model and firmware, and execute NPU-accelerated object detection inference on the Raspberry Pi.

4.1 Object detection

This example performs object detection inference using pre-trained and pre-compiled model parameters provided with the sample package.

The model is designed to detect persons, vehicles, and license plates, and runs inference directly using the Tachy-Shield without requiring any additional training or model compilation steps.

4.1.1 Download Files

First, download the official example repository

```
git clone https://github.com/Deeper-I/TACHY-Examples.git
```

Navigate to the sample application directory and Download the Required api whl, model and firmware binaries into the current directory.

```
cd TACHY-Examples/host_inference_examples/examples/example1
```

SPL: <https://gofile.me/5NFjK/4iUNTeqqE>

U-Boot: <https://gofile.me/5NFjK/dRkyxLi1d>

Image: <https://gofile.me/5NFjK/CA5F0acpY>

FPGA Bits: <https://gofile.me/5NFjK/574JXkL1R>

Python Runtime API (.whl file): <https://gofile.me/5NFjK/9a16ln5LV>

Model File: <https://gofile.me/5NFjK/8pjoLs9Ss>

After placing all required files in the directory, run the following command to ensure the files are present:

```
ls -al
```

You should see entries similar to the following:

```
class.json
fpga_top.bin
image.ub
main.py
model_160x288x3_inv-f.tachyrt
post_configs.json
post_process.py
spl.bin
tachy_rt-3.2.2-py3-none-any.whl
```



```
u-boot.bin
```

4.1.2 Set Up the Environment

Before running the inference example, activate or create the Python virtual environment located at the project root:

```
python3 -m venv ../../venv  
source ../../venv/bin/activate
```

Install the required Python packages:

```
pip install ./tachy_rt-3.2.2-py3-none-any.whl  
pip install packaging
```

Grant access permission to the Tachy-Shield device node:

```
sudo chown ${USER}:${USER} /dev/tachy-bs-0-*
```

4.1.3 Execute

Run the sample inference application:

```
python3 main.py \  
  --model_path "/model_160x288x3_inv-f.tachyrt" \  
  --path_firmware "." \  
  --post_config_path "/post_configs.json"
```

When the sample application starts, you should see log messages similar to the following:

```
[2025-12-11 21:06:20 system_function.py -> upload_firmware(): 71] INFO: write spl  
[2025-12-11 21:06:20 system_function.py -> upload_firmware(): 85] INFO: write uboot  
[2025-12-11 21:06:21 system_function.py -> upload_firmware(): 89] INFO: write kernel  
[2025-12-11 21:06:28 system_function.py -> upload_firmware(): 93] INFO: write fpga  
[2025-12-11 21:06:28 system_function.py -> upload_firmware(): 97] INFO: write jump signal  
[2025-12-11 21:06:28 system_function.py -> upload_firmware(): 101] INFO: reset runtime done value  
[2025-12-11 21:06:28 system_function.py -> wait_for_runtime(): 112] INFO: waiting until runtime init done..  
[2025-12-11 21:06:46 system_function.py -> _boot(): 138] INFO: Success to boot. Check the status via uart or other api  
Success to boot. Check the status via uart or other api  
Sending: [#####] 1/1  
Sending: [#####] 1/1  
[2025-12-11 21:06:46 sender.py -> _send_spi(): 262] INFO: waiting reply  
Receiving: [#####] 1/1  
Receiving: [#####] 1/1  
[2025-12-11 21:06:46 tachyrt_model.py -> __init__(): 17] INFO: init tachyrt_model v2.0  
Sending: [#####] 1/1  
Sending: [#####] 18/18  
[2025-12-11 21:06:51 sender.py -> _send_spi(): 262] INFO: waiting reply  
Receiving: [#####] 1/1  
Receiving: [#####] 1/1
```

```
Sending: [#####] 1/1
Sending: [#####] 1/1
[2025-12-11 21:06:54 sender.py -> _send_spi(): 262] INFO: waiting reply
Receiving: [#####] 1/1
Receiving: [#####] 1/1
Sending: [#####] 1/1
Sending: [#####] 1/1
[2025-12-11 21:06:54 sender.py -> _send_spi(): 262] INFO: waiting reply
Receiving: [#####] 1/1
Receiving: [#####] 1/1
(675, 8) 1
Detected Object : 0

(675, 8) 1
Detected Object : 1
Class : Person
Box : 27.20214080810547 12.811363220214844 262.6498718261719 146.77847290039062
```

If the sample application has already been executed once and the firmware has been successfully loaded onto the Tachy-Shield, you can skip the firmware upload step during subsequent runs by disabling the upload option:

```
python3 main.py \
  --model_path "./model_160x288x3_inv-f.tachyrt" \
  --post_config_path "./post_configs.json" \
  --upload_firmware "false"
```

4.2 ANPR (Automatic Number Plate Recognition)

The ANPR sample is provided as a reference inference example for the Tachy-Shield. It performs stand-alone inference using an image binary with embedded pre-trained model parameters, and supports Korean license plates only.

4.2.1 Download Files

First, download the official example repository

```
git clone https://github.com/Deeper-I/TACHY-Examples.git
```

Navigate to the sample application directory and Download the Required api whl, model and firmware binaries into the current directory.

```
cd TACHY-Examples/standalone_inference_examples/examples/example1
```

*** All files are the same as in Section 4.1 except for the Image binary, which must be replaced with the ANPR-specific Image.**

SPL: <https://gofile.me/5NFjK/4iUNTeqqE>

U-Boot: <https://gofile.me/5NFjK/dRkyxLi1d>

Image: <https://gofile.me/5NFjK/IaG2NKYGT>

FPGA Bits: <https://gofile.me/5NFjK/574JXkL1R>

Python Runtime API (.whl file): <https://gofile.me/5NFjK/9a16ln5LV>

After placing all required files in the directory, run the following command to ensure the files are present:

```
ls -al
```

You should see entries similar to the following:

```
fpga_top.bin
image.ub
main.py
spl.bin
tachy_rt-3.2.2-py3-none-any.whl
u-boot.bin
```

4.2.2 Set Up the Environment

Before running the inference example, activate or create the Python virtual environment located at the project root:

```
python3 -m venv ../../venv
source ../../venv/bin/activate
```

Install the required Python packages:

```
pip install ./tachy_rt-3.2.2-py3-none-any.whl
pip install packaging
```

Grant access permission to the Tachy-Shield device node:

```
sudo chown ${USER}:${USER} /dev/tachy-bs-0-*
```

4.2.3 Execute

Run the sample inference application:

```
python3 main.py \
  --path_firmware "."
```

The initialization process may take approximately 2 minutes to complete. Please wait until the application is fully ready.

When the sample application starts, you should see log messages similar to the following:

```
[2025-12-30 14:26:36 system_function.py -> upload_firmware(): 71] INFO: write spl
[2025-12-30 14:26:36 system_function.py -> upload_firmware(): 85] INFO: write uboot
[2025-12-30 14:26:36 system_function.py -> upload_firmware(): 89] INFO: write kernel
[2025-12-30 14:27:15 system_function.py -> upload_firmware(): 93] INFO: write fpga
[2025-12-30 14:27:15 system_function.py -> upload_firmware(): 97] INFO: write jump
```

```
signal
[2025-12-30 14:27:15    system_function.py -> upload_firmware(): 101]    INFO: reset runtime
done value
[2025-12-30 14:27:15    system_function.py -> wait_for_runtime(): 112]    INFO: waiting until
runtime init done..
[2025-12-30 14:28:10    system_function.py -> _boot(): 138]    INFO: Success to boot. Check
the status via uart or other api
Wait until inference ready
.....
..... done!
Check status
Sending: [#####] 1/1
Sending: [#####] 1/1
[2025-12-30 14:29:50          sender.py ->          _send_spi(): 262]    INFO:
waiting reply
Receiving: [#####] 1/1
Receiving: [#####] 1/1
Sending: [#####] 1/1
Sending: [#####] 1/1
[2025-12-30 14:29:56          sender.py ->          _send_spi(): 262]    INFO:
waiting reply
Receiving: [#####] 1/1
Receiving: [#####] 1/1
Check flag
..... Inference is ready!
----- EVENT -----
PLATE : 12374568
BBOX : [ 842  408 1626  607]
```

Note: The license plate shown in this example output is a sample value for demonstration purposes only. Actual OCR results returned by the application may contain real license plate numbers, and handling of such data is the responsibility of the user.

4.3 Camera Display Example Guide

This guide assumes that the Tachy-Shield has already been successfully booted and initialized. Please complete the boot procedure described in **Section 4.1.3 or 4.2.3 before proceeding**.

4.3.1 Check CSI lane configuration

Depending on the Raspberry Pi model, a CSI lane configuration may be required. Check whether the dummy-csi-sensor overlay is configured with the 2lanes option in /boot/firmware/config.txt.

First, check whether the dummy-csi-sensor overlay is configured with the 2lanes option.

```
sudo grep -n "dtoverlay=dummy-csi-sensor" /boot/firmware/config.txt,
```

If this command produces **no output**, the required configuration is not applied.
In that case, apply the correct CSI lane configuration by running the following command:

```
sudo sed -i '/^dtoverlay=dummy-csi-sensor/d' /boot/firmware/config.txt && \
echo "dtoverlay=dummy-csi-sensor,2lanes" | sudo tee -a /boot/firmware/config.txt
```

Example:

```
dtoverlay=dummy-csi-sensor,2lanes
```

After applying the configuration, reboot the system:

```
sudo reboot
```

4.3.2 Verify camera device

(A) check camera device node

After the overlay and drivers are applied, verify that the camera device nodes are created correctly.

```
ls -al /dev/video*
v4l2-ctl --list-devices
```

Ensure that the expected video device (for example, /dev/video0) is present.

(B) Test video capture using v4l2-ctl

Use `v4l2-ctl` to verify that video data is being received correctly.

```
v4l2-ctl -d 0 --stream-mmap --verbose --stream-count=10
```

Example:

```
VIDIOC_QUERYCAP: ok
    VIDIOC_REQBUFS returned 0 (Success)
    VIDIOC_QUERYBUF returned 0 (Success)
...
    VIDIOC_QBUF returned 0 (Success)
    VIDIOC_STREAMON returned 0 (Success)
cap dqbuf: 0 seq:      0 bytesused: 4147200 ts: 585541.207443 (ts-
monotonic, ts-src-eof)
cap dqbuf: 1 seq:      1 bytesused: 4147200 ts: 585541.240843 delta:
33.400 ms (ts-monotonic, ts-src-eof)
cap dqbuf: 2 seq:      2 bytesused: 4147200 ts: 585541.274245 delta:
33.402 ms (ts-monotonic, ts-src-eof)
cap dqbuf: 3 seq:      3 bytesused: 4147200 ts: 585541.307647 delta:
33.402 ms (ts-monotonic, ts-src-eof)
cap dqbuf: 0 seq:      4 bytesused: 4147200 ts: 585541.341050 delta:
33.403 ms fps: 29.94 (ts-monotonic, ts-src-eof)
cap dqbuf: 1 seq:      5 bytesused: 4147200 ts: 585541.374456 delta:
...

```

Troubleshooting

If no output is shown or the video capture test fails in this step, please check the following:

- Ensure that the Tachy-Shield has been properly booted by completing the steps in **Section 4.1.1–4.1.3 or Section 4.2.1–4.2.3.**
- Verify that the dummy camera driver overlay is correctly applied in `/boot/firmware/config.txt`.
- Confirm that the camera device node (for example, `/dev/video0`) is present after boot.

If any configuration changes are made, reboot the raspberrypi and Tachy-Shield.

4.3.3 Display camera frames using the Python example

First, download the official example repository

```
git clone https://github.com/Deeper-I/TACHY-Examples.git
```

Navigate to the sample application directory and Download the Required api whl, model and firmware binaries into the current directory.

```
cd TACHY-Examples/camera_examples
```

Before running the inference example, activate or create the Python virtual environment located at the project root:

```
source ../venv/bin/activate
```

Install the required Python packages:

```
pip install opencv-python
```

Run the display camera frames example:

```
python3 main.py
```

Appendix

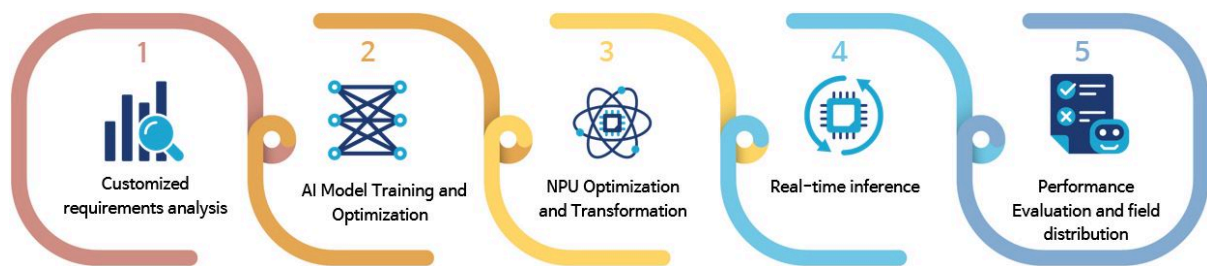
The provided model file allows users to run the sample inference application as-is, demonstrating the standard workflow for the Tachy-Shield and TachyRT runtime.

For commercial projects or custom deployments, additional steps such as model adaptation, TachyRT compilation, post-processing integration, or performance tuning may be required. These advanced features are not included in the sample package and typically depend on the target use-case, dataset, and system configuration.

If technical consultation is required, please contact us for further discussion.

Model Customization & Technical Support

Company Capability Overview



Deeper-I provides a complete end-to-end AI development pipeline based on its proprietary NPU architecture, covering model optimization, compilation, and real-time on-device inference.

We support the full workflow from model training and optimization → TachyRT compilation → NPU-accelerated inference, ensuring high performance, low power consumption, and efficient multi-board operation.

Deeper-I's unique quantization and structural lightweighting method, built on XWN-based technology, delivers superior speed and efficiency tailored for edge AI scenarios.

Our technology enables seamless deployment across smart mobility, smart vision, smart factory, sports analytics, and various other industrial and everyday applications.

Customer Support

please contact us for technical consultation.

We will review your requirements and provide appropriate support or partnership options.

Email: business@deeper-i.ai

Website: <https://www.deeper-i.ai>